Ejercicios: Tema 03

1. Explica qué es un token y qué es un separador. Indica qué función desempeña cada uno en el análisis léxico de un programa.
2. En el siguiente fragmento:

   ```
   int numero = 25;
   System.out.println(numero);
   ```

   Identifica los diferentes tokens que aparecen y clasifícalos como palabra reservada, identificador, literal, operador o separador.
3. Indica qué función cumplen los espacios, tabuladores y saltos de línea desde el punto de vista del análisis léxico de Java. ¿Podría escribirse un programa completo en una sola línea?
4. Indica cuáles de los siguientes elementos son literales:

   ```
   25
   numero
   "25"
   '2'
   true
   false
   null
   MAXIMO
   2.5
   0xFF
   ```

   Clasifícalos además según el tipo de valor que representan.
5. Indica cuáles de los siguientes elementos son palabras reservadas de Java y cuáles pueden ser identificadores:

   ```
   while
   Mientras
   int
   Integer
   class
   Clase
   true
   verdadero
   return
   retorno
   for
   FOR
   ```

6. Explica por qué true, false y null no pueden utilizarse como identificadores.
7. Según la plantilla sintáctica de Identifier, indica si los siguientes identificadores son válidos en Java:

```
numero
2numero
numero2
_numero
numero_2
true
null
class
miNumero
mi numero
```

Justifica los casos inválidos.

8. Observa la siguiente plantilla sintáctica:

```
Literal:
  IntegerLiteral
  FloatingPointLiteral
  BooleanLiteral
  CharacterLiteral
  StringLiteral
  NullLiteral
```

¿De cuántas alternativas dispone Literal? ¿Qué significa que aparezcan varias alternativas?

9. Interpreta la siguiente producción:

```
IntegerLiteral:
  DecimalNumeral
  HexNumeral
  OctalNumeral
  BinaryNumeral
```

¿Qué cuatro formas de representación de un entero permite?

10. Indica cuáles de los siguientes literales enteros son válidos en Java y, cuando sean válidos, qué valor representan:

```
26
026
0x1A
0b11010
0X1a
012
08
0b102
1_000_000
1__000
```

11. Indica qué diferencia existe entre `5`, `5.0` y `5L` desde el punto de vista del tipo del literal.
12. Indica qué tipo tendrá cada uno de los siguientes literales:

```
10
10L
10.0
10.0f
true
'A'
" A "
null
```

13. Indica cuáles de las siguientes construcciones son expresiones y qué tipo de valor producen al evaluarse:

```
5 + 3
5 > 3
!true
numero
numero = 5
```

14. Calcula el resultado de las siguientes expresiones suponiendo que todas las variables son de tipo int:

```
10 + 3 * 2
(10 + 3) * 2
18 / 5
18 % 5
7 + 4 * 3 - 2
20 / 4 * 3
20 / (4 * 3)
```

15. Repite el ejercicio anterior indicando además qué operadores se han aplicado y en qué orden.
16. Indica si las siguientes expresiones producen un resultado entero, real o booleano:

```
7 / 2 7.0 / 2 7 % 2 7 > 2 7 == 2 7 + 2.0
```

17. Explica la diferencia entre los operadores de igualdad (==) y asignación (=). ¿Qué problema produciría confundirlos en una selección?
18. Indica cuáles de las siguientes expresiones son válidas en Java:

```
a < b
a <= b
a == b
a = b
a && b
a < b < c
```

Supón que a, b y c son variables de tipo int.

19. Supón que a y b son variables de tipo boolean. Indica cuáles de las siguientes expresiones son válidas:

```
a == b
a != b
a < b
```

```
a && b
a || b
```

20. Completa la tabla de verdad de:

```
!p
p && q
p || q
```

    para todos los valores posibles de p y q.
21. Sin ejecutar el programa, indica qué expresiones se evalúan cuando x vale 5:

```
if (x < 0 && obtenerDato())
...
```

    y cuando x vale -5. Explica la respuesta teniendo en cuenta la evaluación en cortocircuito.
22. Haz lo mismo con:

```
if (x > 0 || obtenerDato())
...
```

    para x = 5 y x = -5.
23. Explica por qué el orden de los operandos de && y || puede ser importante en Java.
24. Calcula el resultado de las siguientes expresiones teniendo en cuenta la precedencia de los operadores:

```
3 + 4 * 2
10 - 6 / 2
2 + 3 * 4 > 10
5 > 2 && 8 < 10
5 > 2 || 8 > 10
! (5 > 2)
```

25. Añade paréntesis a las siguientes expresiones para dejar explícito el orden de evaluación que corresponde a las reglas de precedencia:

```
a + b * c
a < b && c < d
a + b > c * d
a == b || c != d && e > f
```

26. Explica qué significa que dos operadores tengan la misma precedencia y cómo interviene la asociatividad para determinar el orden de evaluación.
27. Considera:

```
int a = 4;
int b = 9;
```

```
int c = 2;
```

Indica qué valor queda almacenado en a después de cada una de las siguientes sentencias, considerándolas independientes:

```
a = b;
a = a + b;
a = a * c + b;
a = b % c;
```

28. Indica qué ocurre con el tipo de resultado en las siguientes expresiones:

```
int + int
int + long
int + float
long + double
```

29. Explica qué significa que un operando sea promocionado a un tipo más amplio durante la evaluación de una expresión.
30. Indica qué tipo tendrá cada resultado:

```
int a = 5;
double b = 2.0;
a + b
a * b
a / b
```

31. Explica qué ocurre al convertir explícitamente:

```
double x = 7.8;
int y = (int) x;
```

¿Qué valor contiene y?

32. Calcula los resultados de:

```
(int) 7.9
(int) -7.9
(double) 7
(char) 65
```

33. Explica por qué una conversión explícita puede provocar pérdida de información.
34. Indica qué clase envolvente corresponde a cada tipo primitivo:

```
byte
short
int
long
float
double
char
boolean
```

35. Identifica qué operaciones corresponden a *boxing* y cuáles a *unboxing*:

```
Integer n = 5;
int m = n;
Double x = 4.5;
double y = x;
```

36. Indica qué clase envolvente utilizarías para transformar el texto `"123"` en un `int` y qué método utilizarías para ello.
37. Explica qué diferencia conceptual existe entre convertir un `String` a un valor numérico y realizar un *cast* entre tipos numéricos.
38. ¿Qué problema resuelve la biblioteca estándar de Java? ¿Qué relación existe entre una biblioteca, un paquete, una clase y un método?
39. Explica qué significa la siguiente expresión:

```
Math.sqrt(25)
```

    Identifica la clase, el método y el argumento.
40. Indica qué elemento proporciona cada una de las siguientes expresiones:

```
Math.PI
Math.abs(-8)
Math.pow(2, 3)
Math.round(3.6)
Math.floor(3.6)
```

41. Indica qué resultado produce:

```
(int) (Math.random() * 10)
```

    ¿Qué valores enteros puede producir?
42. ¿Por qué la expresión anterior no puede producir el valor `10`?
43. Escribe, utilizando la fórmula estudiada en el tema, una expresión que produzca un entero aleatorio entre `20` y `50`, ambos incluidos.
44. Explica qué problema resuelve una importación estática como:

```
import static java.lang.Math.sqrt;
```

    ¿Cómo cambiaría una llamada a `sqrt`?
45. Explica la diferencia entre una expresión y una sentencia.
46. Clasifica como expresión o sentencia:

```
a + b
a = b
System.out.println(a)
```

```
a > b
```

47. Explica qué es un bloque de código y qué caracteres lo delimitan en Java.
48. Considera:

```
int a = 10;
{
    int b = 20;
    System.out.println(a);
    System.out.println(b);
}
System.out.println(a);
```

    ¿Qué variables pueden utilizarse en cada una de las tres llamadas a `println`?
49. ¿Qué ocurre con una variable declarada dentro de un bloque cuando la ejecución abandona dicho bloque?
50. Explica por qué dos variables pueden tener el mismo nombre si están declaradas en bloques diferentes y sus ámbitos no se solapan.
51. Explica con tus palabras qué es el flujo de control de un programa.
52. Indica la diferencia entre el orden en el que aparecen las sentencias en el código y el orden en el que pueden ejecutarse.
53. Considera:

```
int a = 10;
int b = 20;
int suma = a + b;
System.out.println(suma);
```

    Indica qué instrucciones se ejecutan y en qué orden.
54. Explica por qué una secuencia constituye una estructura de control aunque no cambie explícitamente el flujo lineal de ejecución.
55. Indica qué ocurre con el flujo de control en una selección simple cuando la condición es verdadera y cuando es falsa.
56. Explica la diferencia entre:

```
if ( condicion )
    sentencia;
```

y

```
if ( condicion )
    sentencia1;
else
    sentencia2;
```

57. Indica cuántas sentencias de las siguientes se ejecutan en cada caso:

```
if ( a > 10 )
    System.out.println("A");
System.out.println("B");
```

    para `a = 5` y `a = 20`.
58. Explica qué problema puede producir el siguiente fragmento:

```
if ( a > 0 );
    System.out.println("Positivo");
```

    ¿Se ejecutará siempre la segunda sentencia?
59. Reescribe el siguiente código utilizando una selección compuesta:

```
if ( nota >= 5 )
    System.out.println("Aprobado");
```

    para que también se muestre `"Suspenso"` cuando corresponda.
60. Indica qué camino seguirá el siguiente programa si `nota` vale `8`:

```
if ( nota >= 9 )
    System.out.println("Sobresaliente");
else if ( nota >= 7 )
    System.out.println("Notable");
else if ( nota >= 5 )
    System.out.println("Aprobado");
else
    System.out.println("Suspenso");
```

61. ¿Cuántas condiciones se evalúan si `nota` vale `8`? ¿Y si vale `3`?
62. Explica por qué la estructura anterior puede considerarse una selección encadenada aunque no exista una sentencia `if-else-if` independiente en la gramática de Java.
63. Reescribe conceptualmente mediante un operador ternario:

```
if ( edad >= 18 )
    tipo = "mayor";
else
    tipo = "menor";
```

64. Explica por qué el operador ternario es una expresión y no debe utilizarse para sustituir indiscriminadamente a cualquier `if-else`.
65. Indica a qué `if` pertenece cada `else` del siguiente fragmento:

```
if ( a > 0 )
    if ( b > 0 )
        System.out.println("A");
    else
        System.out.println("B");
```

Explica la regla que utiliza Java.

66. Indica para qué tipo de problema resulta especialmente apropiado un `switch` frente a una secuencia de `if-else-if`.
67. En el siguiente `switch`, indica qué se imprime si `dia` vale `2`:

```
switch ( dia )
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    default:
        System.out.println("Otro");
}
```

68. Explica qué es el *fall-through* de un `switch` y cómo puede utilizarse deliberadamente para agrupar varios `case`.
69. Indica qué restricciones deben cumplir las constantes utilizadas en las etiquetas `case`.
70. Explica la diferencia entre una sentencia `switch` tradicional y una *switch expression*.
71. ¿Qué función desempeñan `->` y `yield` en una *switch expression*?
72. Explica qué diferencia fundamental existe entre una selección y una repetición respecto al flujo de control.
73. Indica qué es una iteración.
74. Considera:

```
int numero = 1;

while ( numero <= 5 )
    System.out.println(numero);
    numero = numero + 1;
}
```

Haz una traza en papel indicando el valor de `numero` y el resultado de la condición en cada comprobación.

75. ¿Cuántas iteraciones realiza el siguiente bucle?

```
int numero = 10;

while ( numero < 5 )
    numero++;
```

76. Explica por qué un `while` puede realizar cero iteraciones.
77. Explica qué problema presenta el siguiente código:

```
int numero = 1;

while ( numero <= 5 )
    System.out.println(numero);
```

78. ¿Es necesariamente un error sintáctico un bucle infinito? Razona la respuesta.
79. Explica qué caracteriza a un bucle controlado por condición.
80. Diferencia entre:
    1. bucle controlado por valor centinela
    2. bucle controlado por bandera
    3. bucle controlado por fin de flujo
81. Indica qué valor centinela elegirías para una aplicación que lea edades válidas entre `0` y `120`.
82. Explica por qué el centinela debe poder distinguirse inequívocamente de los datos válidos.
83. Indica qué representa la variable `finalizado` en un bucle controlado por bandera.
84. Explica la diferencia fundamental entre `while` y `do-while`.
85. Indica cuál de los dos utilizarías en cada situación:
    1. Leer datos hasta encontrar un `0`, pudiendo ser el primer dato el propio `0`.
    2. Solicitar al usuario una nota hasta que esté entre `0` y `10`.
    3. Mostrar números mientras un contador siga dentro de un intervalo.
86. Convierte conceptualmente el siguiente `while` en un `do-while`, sin cambiar el significado:

```
while ( dato < 1 || dato > 10 )
    dato = teclado.nextInt();
```

    ¿Qué condición inicial necesitas para que ambas estructuras tengan realmente el mismo comportamiento?
87. Explica por qué una entrada de datos que debe realizarse necesariamente una primera vez es un ejemplo natural para `do-while`.
88. Enumera los cuatro componentes fundamentales de un bucle controlado por contador.
89. Identifica esos cuatro componentes en:

```
for ( int i = 1; i <= 10; i++ )
    System.out.println(i);
```

90. Escribe la forma equivalente mediante `while`:

```
for ( int i = 1; i <= 10; i++ )
    System.out.println(i);
```

91. Explica qué parte de un `for` se ejecuta una única vez, cuál antes de cada iteración y cuál después de cada iteración.
92. Haz una traza del siguiente bucle:

```
for ( int i = 2; i <= 10; i += 2 )
    System.out.println(i);
```

93. Escribe los valores que toma el contador en:

```
for ( int i = 10; i >= 1; i -= 2 ) {
    ...
}
```

94. Explica qué error produciría intercambiar accidentalmente el sentido de actualización del contador respecto de la condición.
95. ¿Qué es un error *off-by-one*? Pon un ejemplo sencillo relacionado con un intervalo del 1 al 10.
96. Escribe tres formas equivalentes, cuando se utilizan como sentencias independientes, de incrementar `contador` en una unidad.
97. Indica qué diferencia existe entre:

```
contador++;
++contador;
```

    cuando aparecen como sentencias independientes.
98. Explica la diferencia entre:

```
int a = contador++;
```

    y

```
int a = ++contador;
```

99. Haz una traza de:

```
int contador = 5;
int a = contador++;
int b = ++contador;
```

indicando el valor de `contador`, `a` y `b`.

100. Haz lo mismo con los operadores de decremento:

```
int contador = 5;
int a = contador--;
int b = --contador;
```

101. Completa las equivalencias:

```
contador += 5
contador -= 3
contador *= 2
contador /= 4
contador %= 3
```

utilizando solamente el operador `=`.

102. Indica qué valor queda en `x` después de cada una de las siguientes sentencias, suponiendo inicialmente `x = 10`:

```
x += 5;
x -= 3;
x *= 2;
x /= 4;
x %= 3;
```

`Considera cada caso de forma independiente.`

103. Explica por qué `contador += 1` y `contador++` pueden producir el mismo estado final cuando se utilizan como sentencias independientes, aunque no sean la misma construcción sintáctica.
104. Elige entre `if`, `if-else`, `switch`, `while`, `do-while` y `for` para cada una de las siguientes situaciones: a. Mostrar un mensaje únicamente si una edad es mayor de edad. b. Elegir una acción entre tres opciones de un menú. c. Leer datos hasta que aparezca un valor especial. d. Mostrar los números del 1 al 100. e. Solicitar una contraseña hasta que sea correcta, debiendo solicitarla al menos una vez.
105. Para cada una de las situaciones anteriores, explica por qué las demás estructuras serían menos apropiadas.
106. Haz sobre el papel la traza completa del siguiente programa:

```
int i = 1; int suma = 0;
```

```
while ( i <= 4 ) {
    suma += i;
    i++;
}

System.out.println(suma);
</pre>
Indica los valores de `i` y `suma` antes y después de cada iteración.
```

1. Haz una traza del siguiente programa e indica exactamente qué se imprime:

```
 for ( int i = 1; i <= 3; i++ ) {
for ( int j = 1; j <= i; j++ )
    System.out.print(j + " ");
System.out.println();
 }
```

2. Explica, sin escribir código, qué estructura o combinación de estructuras utilizarías para resolver cada uno de los siguientes algoritmos:

```
a. Obtener el mayor de tres valores.
b. Mostrar todos los números pares entre dos valores.
c. Leer una serie de valores hasta encontrar un valor especial.
d. Mostrar una tabla de multiplicar.
e. Clasificar una calificación en cuatro categorías.
f. Repetir una solicitud hasta que el dato sea válido.
```

1. Observa el siguiente código y localiza cualquier posible problema lógico relacionado con el flujo de control:

```
  int i = 1;

while ( i <= 10 ) {
    System.out.println(i);
    i--;
}
</pre>
Explica qué ocurre y por qué.
```

1. Explica qué elementos de un bucle debes comprobar sistemáticamente cuando realizas una revisión manual de un algoritmo repetitivo.
2. Explica la diferencia entre **verificación** y **depuración**. ¿Por qué el hecho de que un programa compile correctamente no garantiza que resuelva el problema para el que se ha escrito?
3. El siguiente programa debería calcular la suma de los enteros comprendidos entre 1 y 5, ambos incluidos. Sin ejecutarlo, realiza una traza y determina qué valor obtiene realmente. Localiza el primer punto en el que la ejecución deja de seguir el proceso esperado y corrige el código.

```
  int numero = 1; int suma = 0;
```

```
while ( numero <= 5 ) {
    numero++;
    suma += numero;
}
System.out.println(suma);
</pre>
```

1. Un programa debe clasificar una nota comprendida entre 0 y 10. Las notas inferiores a 5 son insuficientes; las notas desde 5 hasta menos de 7 son aprobadas; las notas desde 7 hasta menos de 9 son notables; y las notas desde 9 hasta 10 son sobresalientes. Propón un conjunto de datos de prueba que permita comprobar todas las categorías y los valores situados en sus límites. Para cada dato, indica cuál debería ser el resultado.
2. El siguiente fragmento debería sumar los valores de una secuencia. Al probarlo con los datos 3, 4 y 5, el resultado no es el esperado.

```
  int suma = 0; int numero = teclado.nextInt();
```

```
while ( numero != 0 ) {
    suma = numero;
    numero = teclado.nextInt();
}
System.out.println(suma);
</pre>
Realiza una traza para los datos indicados, anota el valor de `suma` después de cad
```

1. Se ha escrito un programa que debe determinar si un entero es primo. Antes de ejecutarlo, diseña una pequeña batería de pruebas que incluya casos normales y casos límite. Para cada entrada propuesta, escribe el resultado que debería proporcionar el programa y explica qué aspecto de la solución permite verificar.
2. Un programa lee enteros hasta que se introduce el cero. Debe calcular cuántos datos se han leído y su suma, sin incluir el cero centinela. Explica qué casos de prueba utilizarías para comprobar que el programa funciona correctamente cuando se introducen varios valores, cuando el primer valor es cero y cuando el cero aparece después de un único dato.
3. Un programa valida una nota y debe aceptar exclusivamente valores comprendidos entre 0 y 10, ambos incluidos. Diseña pruebas para comprobar los dos extremos del intervalo, dos valores interiores y un valor inmediatamente fuera de cada extremo. Indica para cada dato si debe aceptarse o rechazarse.
4. Un programa presenta cuatro alternativas mediante una estructura de selección. Explica qué significa comprobar los diferentes caminos de ejecución. Propón un conjunto de pruebas que permita verificar que se alcanza cada alternativa y que también se trata correctamente una opción no válida.
5. ¿Por qué una traza correcta para un único conjunto de datos no demuestra que un programa sea correcto para todas las entradas? Utiliza como ejemplo un programa

que calcula el factorial y propone datos que permitan comprobar su comportamiento normal, los límites de entrada y algún caso que pueda revelar un error.

6. Al probar un programa, observas que compila y termina normalmente, pero calcula un resultado incorrecto. Describe un procedimiento ordenado para localizar la causa. Incluye en tu explicación la comparación entre el resultado esperado y el obtenido, la elección de datos de prueba y la utilización de una traza para localizar el primer estado incorrecto.
7. El siguiente bucle pretende mostrar los números del 1 al 5 y sumar sus valores, pero contiene varios errores:

```
  int numero = 1; int suma = 0;
```

```
while ( numero <= 5 ); {
    if ( numero = 3 ) {
        suma += numero;
    }
    numero--;
}

System.out.println(suma);
</pre>
Identifica los errores, explica si cada uno es sintáctico, de compilación o lógico
```

Tema3 | Problemas